



High Performance Computing 2026S Assignment 1

Issue date: 2026-04-28

Due date: 2026-05-12

No extensions.

Please do not forget to register with a group on TUWEL! Otherwise, you will not be able to submit your answers.

What to Hand in via TUWEL?

1. A PDF file containing your answers, using the provided report template.
2. Your source code as `bcast.zip`, generated with:

```
bcast$ cmake --build ./build/ --target pack
```

Do not submit RAR, Word, or other formats.

3. Your raw benchmark data for the `bcast` exercise as `data.zip`.

Rules

- Ensure that the following information is provided in the PDF document:
 - names of authors
 - date
 - course and assignment name
 - TUWEL group size and number
 - * Example: If you are registered for “3 Person Group 12” on TUWEL, then your group size is 3 and your group number is 12.
 - Use a spell checker. Check your grammar.
 - Label your figures and tables appropriately, e.g., “Table 1: Summary” is not an appropriate description.
 - Reference each figure and table in the text. Ensure that each figure/table is indeed described in the text. Only stating that “Table 1 contains a summary.” is not considered appropriate. Please explain what exactly is summarized in a table.
-

Part 1: The Roofline Model

Exercise 1 (10 [2*(2+1+2)] points): Obtain Roofline Model for Two Machines

Obtain a Roofline model for **two machines**:

- one multicore machine **of your choice** (e.g., your laptop or a lab workstation), and
- one compute node of the hydra cluster (hydra01–hydra36; do **not** use the frontend node hydra-head).

Perform the following steps for **each machine** (points below refer to one machine):

1. Using the processor specification:
 - calculate the **theoretical peak performance** (GFLOP/s) using all physical cores, and
 - run the BabelStream benchmark (on all cores) to **measure the peak memory bandwidth**. Use the Triad kernel to obtain the memory bandwidth, and ensure the total array size exceeds the last-level cache (e.g., use the default BabelStream array size or increase it until the measured bandwidth stabilizes).

(2 points)

For computing the theoretical peak performance, you need to find out the FLOPS/cycle of the processor, which depends on the supported instruction sets and the number of floating-point units. Also, ensure to use the correct clock frequency for the calculation, as the turbo boost frequency may not be sustained when all cores are active.

2. Document the **compiler flags and parameters** used to build BabelStream, and include the **full benchmark output**. The benchmark output should be included in the appendix of your submission report. (1 point)
3. Plot a **Roofline model** for the machine as introduced in the lecture. Label axes, the memory-bandwidth roof, and the compute roof correctly. (2 points)

To build BabelStream and hydra, you can use the following commands:

```
$ git clone https://github.com/UoB-HPC/BabelStream
$ cd BabelStream
$ spack load cmake@3.26.3
$ git checkout v5.0
$ cmake -B build -DMODEL=omp
$ cmake --build build -j
```

Exercise 2 (10 points): Determine the Arithmetic Intensity (AI) of a Compute Kernel

In this exercise you will empirically determine the **arithmetic intensity (AI)** of a stencil kernel using hardware performance counters, and place it on the Roofline model from the previous exercise.

Setup. Use the OPENMP/Stencil kernel from the Parallel Research Kernels (PRK) repository and the LIKWID performance tool suite, which is available on hydra via the module system.

```
~/hpc$ git clone https://github.com/ParRes/Kernels
~/hpc$ cd Kernels/OPENMP/Stencil
~/hpc/Kernels/OPENMP/Stencil$ cp ../../common/make.defs.gcc ../../common/make.
defs
```

Change the gcc version in common/make.defs:

```
-VERSION=-14
+VERSION=
```

Then you can compile the kernel with make, e.g.:

```
# star stencil
$ make STAR=1 stencil
# compact stencil
$ make STAR=0 stencil
```

Task 1: Whole-program AI. Use LIKWID to measure the floating-point operation count and the number of bytes transferred to/from main memory for the **entire program**. Run all combinations shown in Table 1 on a hydra compute node.

Table 1: Parameters for measuring the arithmetic intensity of the stencil kernel.

Parameter	Values
Array dimension	1000, 50000
Iterations	2, 10
Stencil type	star, compact
Thread count	2, 16, 32

Plot all resulting AI values on the Roofline model of the hydra node obtained in the previous exercise. Provide four plots, one for each combination of array dimension and stencil type. For each plot, use different markers for the different thread counts and iterations, i.e., iterations, thread count = (2,2), (2,16), (2,32), (10,2), (10,16), (10,32). Discuss the results: explain where the measurements fall on the Roofline model and why. (4 points)

To measure the whole program (Task 1), run likwid-perfctr without the marker flag:

```
$ srun -N1 likwid-perfctr -C 0-31 -g FLOPS_DP ./stencil 2 2 1000
$ srun -N1 likwid-perfctr -C 0-31 -g MEM ./stencil 2 2 1000
```

Task 2: Kernel-only AI. Instrument the source code with **LIKWID marker calls** to measure only the innermost stencil loop (i.e., excluding initialization and other overhead). Repeat the measurements with the values given in Table 1 using the marker-scoped counters and create four additional Roofline plots showing the AI of the stencil kernel iterations exclusively. Discuss the

results: compare the AI of the whole program and the kernel-only measurements, and explain any differences. (6 points)

To compile the stencil kernel with the LIKWID Marker API, add the following lines to the kernel's Makefile:

```
USERFLAGS = -DLIKWID_PERFMON
LIBS = -llikwid
LIBPATHS = -L/opt/spack/spack_git_updated/opt/spack/linux-debian11-skylake_avx512/gcc-10.2.1/likwid-5.2.2-
           eljbkkyevbvcpln4hn6u2nmsnz76x2y/lib -Wl,-rpath,/opt/spack/spack_git_updated/opt/spack/linux-debian11-
           skylake_avx512/gcc-10.2.1/likwid-5.2.2-eljbkkyevbvcpln4hn6u2nmsnz76x2y/lib
INCLUDEPATHS = -I/opt/spack/spack_git_updated/opt/spack/linux-debian11-skylake_avx512/gcc-10.2.1/likwid-5.2.2-
               eljbkkyevbvcpln4hn6u2nmsnz76x2y/include
```

Remove the linebreaks in the LIBPATHS and INCLUDEPATHS variables when copying the code into the Makefile. You can also get this information from the LIKWID module on hydra:

```
$ spack load likwid@5.2.2
$ which likwid-perfctr
/opt/spack/spack_git_updated/opt/spack/linux-debian11-skylake_avx512/gcc-10.2.1/
  likwid-5.2.2-eljbkkyevbvcpln4hn6u2nmsnz76x2y/bin/likwid-perfctr
```

We suggest to create a copy of the original stencil kernel, e.g., `stencil_likwid.cc`, and add the LIKWID marker calls to the copy. This way, you can easily compare the results of Task 1 and Task 2. To this end, you will have to modify the Makefile to add a new build target for `stencil_likwid`. Ensure to insert the correct LIKWID marker calls to the innermost stencil loop in `stencil_likwid.cc`. To measure only the instrumented region (Task 2), add the `-m` flag:

```
$ srun -N1 likwid-perfctr -m -C 0-31 -g FLOPS_DP ./stencil_likwid 2 2 1000
$ srun -N1 likwid-perfctr -m -C 0-31 -g MEM ./stencil_likwid 2 2 1000
```

Part 2: MPI Collective Communication

In this part you will implement two broadcast algorithms and evaluate their performance. Your implementations use the **PMPI profiling interface**: by providing a custom `MPI_Bcast` symbol that internally calls `PMPI_Bcast` (or your own point-to-point code), you can transparently replace the library's broadcast without modifying application code.

Prepare your environment on hydra by loading the required packages:

```
$ spack load openmpi@4.1.6
$ spack load gcc@13.2.0
$ spack load cmake@3.26.3
```

Building the Implementations. Download the skeleton code for the broadcast implementations from the course repository (`bcast.zip`). For each exercise, there is a skeleton file inside the `bcast/` directory. Build the shared library with CMake:

```
bcast$ cmake -B ./build
bcast$ cmake --build ./build
```

Testing the Implementations. Test the correctness of your implementations with the provided test suite:

```
stester@hydra-head:~/hpc/bcast$ srun -t 1 -p q_student -N1 --ntasks-per-node=2 ./
  build/bin/bcast_test -m 100
PASS size=100 bytes np=2
stester@hydra-head:~/hpc/bcast$ srun -t 1 -p q_student -N1 --ntasks-per-node=2 ./
  build/bin/bcast_test -m 101
PASS size=101 bytes np=2
```

Of course, you can vary the message size and the number of processes to test more configurations.

Benchmarking with ReproMPI. Use ReproMPI to benchmark your implementations against the MPI library's default MPI_Bcast. Build ReproMPI once inside the bcast/ directory (the benchmark commands assume this location):

```
bcast$ git clone https://github.com/hunsa/reprompi-dev.git
bcast$ cd reprompi-dev
reprompi-dev$ cmake -B ./build
reprompi-dev$ cmake --build ./build
```

Then inject your library at runtime via LD_PRELOAD (see the individual exercises for the exact invocations).

Exercise 3 (5 points): Implement MPI_Bcast with Knomial Tree Algorithm

Implement a k -nomial tree broadcast in bcast_knomial.cc. The branching factor k must be configurable at runtime via the environment variable HPC_BCAST_KNOMIAL_K.

If you want to test the current performance of your implementation, you can use the following command:

```
$ HPC_BCAST_KNOMIAL_K=3 LD_PRELOAD=./bcast/build/libbcast_knomial.so srun -N 1
  --ntasks-per-node=32 reprompi-dev/build/bin/mpibenchmark --msizes-list=1000 --
  nrep=1 --calls-list=MPI_Bcast
```

Exercise 4 (5 points): Implement MPI_Bcast with Chain Algorithm

Implement a segmented chain broadcast in bcast_chain.cc. Two environment variables control the behaviour:

- HPC_BCAST_CHAIN_NUM — number of parallel chains,
- HPC_BCAST_CHAIN_SEG — segment size (in bytes) per chain.

To benchmark your implementation:

```
$ HPC_BCAST_CHAIN_NUM=2 HPC_BCAST_CHAIN_SEG=10 LD_PRELOAD=./bcast/build/
  libbcast_chain.so srun -N 1 --ntasks-per-node=32 reprompi-dev/build/bin/
  mpibenchmark --msizes-list=1000 --nrep=1 --calls-list=MPI_Bcast
```

Exercise 5 (12 points): Evaluate the Performance of your Implementations

Exercise 5.1 (8 points): Compare Implementations

Benchmark both of your implementations against the MPI library's default MPI_Bcast using Re-proMPI on the hydra cluster.

Use the following **message sizes** (in bytes):

1, 10, 100, 1 000, 10 000, 100 000, 1 000 000, 10 000 000, 100 000 000

Run each configuration for the following **process layouts** (nodes $\times$ processes per node):

Nodes	Processes per node
1	32
16	1
16	2
16	32

For the chain algorithm, fix the parameters at HPC_BCAST_CHAIN_NUM=2 and HPC_BCAST_CHAIN_SEG=1024. For the knomial algorithm, fix the branching factor at HPC_BCAST_KNOMIAL_K=2. Use the batch job file `bcast/jobs/mpi_job.sh` to run the benchmarks. Adjust the `#SBATCH -N` and `#SBATCH -ntasks-per-node` directives to match each process layout, and update `HOMEDIR` to point to your home directory before submitting with `sbatch`.

- Plot the results and discuss your findings. Create four plots, one for each process layout, i.e., 1×32 , 16×1 , 16×2 , 16×32 . On the x-axis, use the message size (bytes, logarithmic scale), and on the y-axis, use the median runtime in μ s (microseconds, logarithmic scale).
- Answer the following questions based on your results:
 - Which algorithm performs best for small vs. large messages?
 - How does the number of processes affect the runtime?
 - How does the message size affect the runtime?

Exercise 5.2 (4 points): Parameter Study for the Chain Algorithm

Fix the process layout to **16 nodes $\times$ 32 processes per node** and evaluate the chain algorithm with the parameter combinations given in Table 2. Use the batch job file `bcast/jobs/mpi_job_chain.sh` to run the benchmarks.

Table 2: Parameters for benchmarking the chain algorithm.

Parameter	Values
Message size	1 048 576 Bytes (1 MB), 10 485 760 Bytes (10 MB)
Number of chains	1, 2, 4, 8
Segment size	1 024 Bytes, 5 120 Bytes

There should be a total of 8 configurations (2 segment sizes $\times$ 4 chain numbers) for each message size, i.e., 16 configurations in total.

1. Create two plots, one for each message size, showing the median runtime of the chain algorithm for all parameter combinations and compare these variants against the default MPI_Bcast. Use bars to visualize the different algorithm variants (x and y axes should be in linear scale).
2. Answer the following questions based on your results:
 - Which combination of chains and segment size works best, and why?
 - Are there any measurement outliers? Can you explain them?